# AeroSentinel | Team Astro

## Environment and tasks

A deterministic, simplified CubeSat fault-recovery environment. Six tasks across three difficulty levels cover electrical power, pointing control, thermal regulation, downlink software, a seeded two-subsystem cascade and hard RF contact planning. The demo is eps_eclipse_recovery. State includes battery energy, temperature, scalar pointing error, elapsed time and downloaded data. No agent training is required.

## Actions and observations

Agents inspect files, edit software controls, run bounded Python and advance orbital time by 1-30 minutes. Ordinary actions consume 30 simulated seconds; the mission lasts 95 simulated minutes. Reset telemetry comes from the protected simulator. Mission targets and fixed hardware are exposed in docs/mission.json for planning; editing that file cannot change protected requirements.

## Reward design

The raw mission score combines power (0.35), thermal behavior (0.25), pointing (0.20) and communications (0.20). The reported reward is max(0, (raw - initial raw) / (1 - initial raw)), capped at one. This measures progress over the no-op state. Every required check must pass; failed requirements cap normalized reward below success. Diagnostics identify failed checks and report RF energy use. Hardware safety failures are permanently latched and score zero. The pointing target is protected; final configuration validation runs even at the mission boundary.

## Seeded variation

Seeds vary faults, propagation loss, antenna hardware and EPS energy targets. The hard contact mission varies pass timing, payload and an RF energy allocation: nominal and maximum radio settings both fail, while instance-specific feasible plans pass. Reference configurations retain the instance's actual hardware values. The manual supplies operational guidance and nominal software values; this is intentionally a guided recovery environment, not a claim of zero answer leakage.

## Physics assumptions

The model uses a scalar pointing approximation, linear state-of-charge voltage, linearized thermal loss, a bandwidth-aware link-margin equation and baud-limited data transfer. Integration splits at contact and sunlight boundaries; RF bus energy is integrated over the actual pass. It is useful for controlled evaluation, not validated flight software or a high-fidelity spacecraft simulator.

## Isolation and reset

Verifier and reward code run only in the harness container. Every tool endpoint requires the harness token. Agent and evaluation code use separate UIDs (1001 and 1002). All worker processes, including detached descendants, are killed after execution and before reset. The sandbox has no external route, a read-only root, bounded workspace/tmp mounts and process, CPU, memory, file and output limits. Protected file access rejects symlinks and hard links.

## Harness contract

One Docker Compose command runs reference and no-op checks before the model. Baselines have a combined 120-second per-task budget. Model execution allows 40 replies or 600 seconds; each reply allows 10 tools and each tool operation is bounded to 60 seconds. A cancellable provider process bounds API calls. Final verification has its own 120-second limit. Model, key and endpoint use the same interface for all providers. Judge append text is preserved and logged.

## Telemetry and reproducibility

JSONL records contain model inputs, raw response text, parsed actions, tool results, errors, reward and state hash. Final records include termination and verifier diagnostics. Manifests fingerprint the harness and sandbox source. Packaging accepts only evidence matching the current code, with complete reply sequences and consistent summaries. No credentials belong in logs or archives.

## Known limitations and tests

The physics, task generator and reward targets remain simplified. UID isolation is not a VM boundary and still relies on Docker and the host kernel. Tests cover baseline thresholds, 120 task/seed combinations plus 100 separate shortcut checks, malformed replies, context pressure, timeout recovery, hardware edits, post-orbit invalid configurations, and live container process/resource isolation. Real-provider results and clean-build timing must be taken from supplied logs; no fixed performance claims are embedded here.

## Submission

Submit Astro.zip with all-task baseline results, at least three genuine model runs, success/failure trajectories and these two PDFs. The GitHub repository should match the registered team name and organiser visibility/access requirements. Historical logs remain historical; the packager refuses to manufacture missing evidence.